

Northeast Wildfire Smoke Scraper

Evidence-aware data collection pipeline for cross-border smoke events affecting the Northeast U.S.

Python stdlib

29 output fields

15 source seeds

3/3 tests

PROJECT AT A GLANCE

Collects local or official web pages, extracts readable article text, builds structured wildfire-smoke records, keeps Northeast-relevant events, removes duplicates, and exports CSV, JSON, and an HTML report. Rules mode is deterministic and dependency-free; optional AI mode uses the OpenAI Responses API for difficult fields while preserving program-controlled provenance fields.

IMPACT AND VALUE

- Converts dispersed official reports into a consistent 29-column research dataset and shareable report.
- Reduces repeated downloads through hashing/cache support and respects sources with rate limiting and timeouts.
- Uses Not Reported / Not Confirmed safeguards so missing evidence is not silently turned into a claim.
- Ships an offline fixture and demo outputs, allowing repeatable review without internet or API credentials.

LANGUAGES AND STACK

Languages/data: Python, JSON, HTML/CSS, TOML, CSV, VS Code JSON.
Standard library: urllib, HTMLParser, argparse, dataclasses, csv/json, unittest.
Optional integration: OpenAI Responses API over HTTPS; no required third-party packages.

CODE FLOW - TOP TO BOTTOM



FILE MAP - 29 FILES

Inventory: maintained tree plus shipped demo outputs; excludes .env, caches, and runtime download cache.

.env.example
Shows optional OpenAI API key and model environment variables without storing a secret.

.vscode/launch.json
Defines debugger launches for the offline scraper demo and unittest discovery.

.vscode/tasks.json
Adds terminal tasks for the scraper demo and full test suite.

config/sources.json
Provides 15 seeded official-source events with URLs and evidence hints.

data/demo_events.csv
Two-line demonstration export containing the full 29-column schema and one record.

data/demo_report.html
Browser-ready demonstration report with a concise event table.

README.md
Covers setup, offline/live runs, sources, optional AI, testing, structure, and limitations.

src/wildfire_scraper/_init_.py
Marks and documents the reusable wildfire_scraper package.

src/wildfire_scraper/ai.py
Calls the Responses API, validates JSON, and preserves program-controlled provenance.

src/wildfire_scraper/exporter.py
Serializes events to 29-column CSV, JSON, and escaped HTML table output.

src/wildfire_scraper/html_text.py
Ignores scripts/styles and converts HTML blocks, titles, and headings to clean text.

src/wildfire_scraper/parser.py
Extracts dates, origin, AQI/PM2.5, area, smoke path, impacts, sources, and notes.

tests/_init_.py
Marks the tests directory as a Python package.

tests/test_exporter.py
Checks that CSV export contains every modeled field and expected row values.

WildfireSmokeScraper.code-workspace
Opens the project with src imports and unittest settings preconfigured.

.gitignore
Excludes environments, secrets, caches, and generated data while keeping data/.gitkeep.

.vscode/settings.json
Configures src import resolution, unittest discovery, and cache hiding in VS Code.

config/demo_sources.json
Points the offline demo at the bundled NOAA-style HTML fixture.

data/.gitkeep
Keeps the otherwise generated-data directory in the project package.

data/demo_events.json
Machine-readable version of the demonstration wildfire record.

pyproject.toml
Defines setuptools packaging, Python 3.10+, src layout, and wildfire-scraper console entry.

requirements.txt
Documents that the core project requires only Python's standard library.

src/wildfire_scraper/_main_.py
Routes python -m wildfire_scraper to the CLI main function.

src/wildfire_scraper/cli.py
Defines scrape arguments, runs the pipeline, writes exports, and returns status codes.

src/wildfire_scraper/fetcher.py
Reads local/HTTP pages with cache hashing, user agent, delay, timeout, and refresh.

src/wildfire_scraper/models.py
Defines the 29-field WildfireEvent model, conversions, and Northeast relevance test.

src/wildfire_scraper/pipeline.py
Loads/validates sources, resolves local URLs, fetches/parses/enhances, filters, and dedupes.

tests/fixtures/noaa_sample.html
Small offline NOAA-style article fixture with Quebec, Northeast, AQI, and advisory evidence.

tests/test_parser.py
Tests cross-border extraction and verifies that unsupported border claims stay unconfirmed.